Dưới đây là **toàn bộ cây thư mục hoàn chỉnh**, với chú thích chức năng được **ghi trực tiếp ngay bên cạnh từng file và từng thư mục**:

```
ai-pentest-copilot/
├── .env                                    # Chứa biến môi trường & API keys (OpenAI, Gem
├── .env.example                            # File mẫu cấu hình biến môi trường
├── .github/                                # Cấu hình GitHub Actions
│   └── workflows/
│       └── ai-pentest-scan.yml             # CI/CD workflow tự động quét an ninh & chạy t
├── .gitignore                              # Danh sách file/thư mục Git bỏ qua (cache, ve
├── Makefile                                # Lệnh tắt build, test, setup, scan cho Linux/
├── README.md                               # Tài liệu tổng quan dự án & hướng dẫn sử dụng
├── docker-compose.yml                      # Cấu hình Docker Compose dựng Backend, Fronte
├── make.bat                                # Script tương đương Makefile cho Windows CMD
├── make.ps1                                # Script tương đương Makefile cho Windows Powe
├── package.json                            # Quản lý dependencies Node.js cấp độ root
├── package-lock.json                       # Khóa phiên bản dependencies Node.js
├── run.py                                  # Universal CLI Runner (chạy scan, ingest, tes
├── run_kali.sh                             # Script 1-click khởi chạy & cài đặt tối ưu ch
│
├── backend/                                # PHÂN HỆ BACKEND (Python FastAPI)
│   ├── requirements.txt                    # Danh sách thư viện Python cần cài đặt
│   ├── app/                                # Mã nguồn ứng dụng chính
│   │   ├── __init__.py                     # Khởi tạo package Python
│   │   ├── config.py                       # Đọc và quản lý cấu hình hệ thống từ .env
│   │   ├── main.py                         # Điểm khởi động FastAPI (CORS, Routers, WebSo
│   │   │
│   │   ├── api/                            # Các Router REST API endpoints
│   │   │   ├── chat.py                     # API lịch sử chat & fallback REST cho Copilot
│   │   │   ├── findings.py                 # API quản lý, lọc và xuất danh sách lỗ hổng
│   │   │   ├── health.py                   # API kiểm tra tình trạng sống (healthcheck) h
│   │   │   ├── llm.py                      # API kiểm tra và chuyển đổi LLM provider (Oll
│   │   │   ├── observability.py            # API theo dõi metrics LLM (tokens, latency, t
│   │   │   ├── recon.py                    # API điều phối và theo dõi tiến trình trinh s
│   │   │   ├── reports.py                  # API sinh và tải báo cáo Pentest (MD, HTML, S
│   │   │   ├── scans.py                    # API tạo mới, hủy và xem trạng thái các lượt
│   │   │   ├── targets.py                  # API quản lý mục tiêu (Target) và phạm vi (Sc
│   │   │   ├── terminal.py                 # API thực thi lệnh Terminal (REST & WebSocket
│   │   │   ├── topology.py                 # API cung cấp dữ liệu đồ thị bề mặt tấn công
│   │   │   ├── verification.py             # API tạo và chạy kế hoạch xác minh lỗ hổng an
│   │   │   └── vulnerabilities.py          # API tra cứu cơ sở tri thức CVE, CWE, KEV
│   │   │
│   │   ├── catalog/                        # Danh mục công cụ pentest khai báo dạng YAML
│   │   │   ├── __init__.py                 # Khởi tạo catalog package
│   │   │   └── tools/                      # File định nghĩa cách chạy & regex parse từng
│   │   │       ├── burpsuite.yaml          # Cấu hình Proxy/Manual testing với Burp Suite
│   │   │       ├── database.yaml           # Cấu hình rà soát dịch vụ CSDL (MySQL, Postgr
│   │   │       ├── enum4linux.yaml         # Cấu hình thu thập thông tin Windows/Samba
│   │   │       ├── ffuf.yaml               # Cấu hình fuzzing đường dẫn web siêu tốc
│   │   │       ├── gobuster.yaml           # Cấu hình dò quét thư mục & DNS brute-force
│   │   │       ├── httpx.yaml              # Cấu hình thăm dò dịch vụ HTTP/HTTPS & status
```

```
│   │   │       ├── katana.yaml              # Cấu hình thu thập liên kết web (web crawler/
│   │   │       ├── ldapsearch.yaml          # Cấu hình truy vấn dịch vụ thư mục Active Dir
│   │   │       ├── masscan.yaml             # Cấu hình quét cổng băng thông cực nhanh
│   │   │       ├── naabu.yaml               # Cấu hình quét cổng nhanh tích hợp Nmap
│   │   │       ├── nikto.yaml               # Cấu hình quét điểm yếu cấu hình máy chủ web
│   │   │       ├── nmap.yaml                # Cấu hình quét cổng, phát hiện OS & dịch vụ c
│   │   │       ├── nmap_nse.yaml            # Cấu hình quét lỗ hổng nâng cao bằng Nmap Scr
│   │   │       ├── nuclei.yaml              # Cấu hình quét lỗ hổng theo template dựa trên
│   │   │       ├── rustscan.yaml            # Cấu hình quét cổng siêu tốc viết bằng Rust
│   │   │       ├── searchsploit.yaml        # Cấu hình tìm kiếm mã khai thác Exploit-DB of
│   │   │       ├── smbclient.yaml           # Cấu hình kiểm tra chia sẻ tệp SMB/Windows Sh
│   │   │       ├── snmpwalk.yaml            # Cấu hình trích xuất dữ liệu giao thức quản t
│   │   │       ├── testssl.yaml             # Cấu hình kiểm tra chứng chỉ và cấu hình mã h
│   │   │       ├── theharvester.yaml        # Cấu hình thu thập email, subdomain qua nguồn
│   │   │       ├── whatweb.yaml             # Cấu hình nhận diện công nghệ web (CMS, Serve
│   │   │       └── zap.yaml                 # Cấu hình quét bảo mật web tự động với OWASP
│   │   │
│   │   ├── core/                            # Bộ khung thực thi và an toàn cốt lõi
│   │   │   ├── context.py                   # Quản lý ngữ cảnh phiên làm việc nạp vào LLM
│   │   │   ├── executor.py                  # Dual-mode engine thực thi lệnh (trên OS hoặc
│   │   │   ├── guardrails.py                # Chốt chặn bảo vệ: cấm quét Out-of-Scope, chặ
│   │   │   ├── observability.py             # Thu thập số liệu token, độ trễ và dấu vết su
│   │   │   └── prompts.py                   # Chứa System Prompts định hình năng lực và va
│   │   │
│   │   ├── eval/                            # Thư mục chứa logic đánh giá độ chuẩn xác của
│   │   │
│   │   ├── ingest/                          # Nạp dữ liệu tri thức an ninh vào CSDL SQLite
│   │   │   ├── cwe.py                       # Script kéo & nạp danh mục CWE (Common Weakne
│   │   │   ├── kev.py                       # Script nạp danh mục CISA Known Exploited Vul
│   │   │   ├── mitre.py                     # Script nạp ma trận chiến thuật & kỹ thuật MI
│   │   │   ├── nvd.py                       # Script nạp dữ liệu CVE từ National Vulnerabi
│   │   │   └── run.py                       # Runner điều phối toàn bộ quá trình nạp tri t
│   │   │
│   │   ├── schemas/                         # Định nghĩa cấu trúc dữ liệu Pydantic
│   │   │   ├── __init__.py                  # Khởi tạo schemas package
│   │   │   ├── cve.py                       # Schema mô tả thông tin CVE (ID, CVSS, mô tả)
│   │   │   ├── cwe.py                       # Schema mô tả điểm yếu phần mềm CWE
│   │   │   ├── finding.py                   # Schema biểu diễn lỗ hổng phát hiện (bằng chứ
│   │   │   ├── graph.py                     # Schema định nghĩa các Node và Edge của Topol
│   │   │   ├── target.py                    # Schema định nghĩa mục tiêu (IP, domain, port
│   │   │   └── verification.py              # Schema kế hoạch xác minh lỗ hổng không phá h
│   │   │
│   │   ├── services/                        # Tầng xử lý nghiệp vụ & dịch vụ tích hợp
│   │   │   ├── __init__.py                  # Khởi tạo services package
│   │   │   ├── compliance_engine.py         # Ánh xạ lỗ hổng sang tiêu chuẩn ISO 27001, NI
│   │   │   ├── correlator.py                # Tương quan hóa dữ liệu từ nhiều tool vào cùn
│   │   │   ├── findings_builder.py          # Tổng hợp chuỗi bằng chứng (evidence chain) t
│   │   │   ├── graph_builder.py             # Xây dựng đồ thị bề mặt tấn công từ kết quả s
│   │   │   ├── llm.py                       # Giao tiếp LLM (OpenAI, Ollama, Groq, streami
│   │   │   ├── rag.py                       # Retrieval-Augmented Generation truy vấn tri
│   │   │   ├── recon_planner.py             # Lập kế hoạch trinh sát theo từng bước phù hợ
│   │   │   ├── report_generator.py          # Xuất báo cáo chuyên nghiệp định dạng Markdow
│   │   │   ├── sarif_exporter.py            # Xuất kết quả phát hiện ra chuẩn công nghiệp
```

```
│   │   │   ├── scope_validator.py          # Xác thực nghiêm ngặt mục tiêu có nằm trong p
│   │   │   ├── target_normalizer.py        # Chuẩn hóa định dạng mục tiêu (loại bỏ schema
│   │   │   ├── target_parser.py            # Bóc tách mục tiêu thành IP, Hostname, Port,
│   │   │   ├── target_registry.py          # Quản lý danh bạ mục tiêu trong phiên kiểm th
│   │   │   ├── target_validator.py         # Kiểm tra tính hợp lệ của IP, CIDR, Domain
│   │   │   ├── tool_registry.py            # Đọc và quản lý danh sách công cụ từ file YAM
│   │   │   ├── verification_planner.py     # Tạo kịch bản xác thực lỗ hổng ở chế độ Read-
│   │   │   └── agents/                     # Hệ thống Multi-Agent chuyên biệt hóa
│   │   │       ├── __init__.py             # Khởi tạo agents package
│   │   │       ├── orchestrator.py         # Agent điều phối: hiểu lệnh tự nhiên & chia v
│   │   │       ├── recon_analyst_agent.py  # Sub-Agent chuyên sâu phân tích kết quả trinh
│   │   │       ├── threat_intel_agent.py   # Sub-Agent chuyên tra cứu thông tin tình báo
│   │   │       └── verification_agent.py   # Sub-Agent lập kế hoạch xác minh lỗ hổng an t
│   │   │
│   │   ├── storage/                        # Tầng lưu trữ cơ sở dữ liệu SQLite
│   │   │   ├── chat_db.py                  # Lưu lịch sử tin nhắn trò chuyện với Copilot
│   │   │   ├── db.py                       # Quản lý kết nối SQLite dùng chung
│   │   │   ├── findings_db.py              # Lưu trữ và lọc trùng các lỗ hổng đã phát hiệ
│   │   │   └── graph_db.py                 # Lưu trữ các đỉnh và liên kết của đồ thị bề m
│   │   │
│   │   ├── utils/                          # Các hàm tiện ích dùng chung
│   │   └── ws/                             # Giao tiếp thời gian thực WebSocket
│   │       └── chat.py                     # Xử lý kết nối WebSocket cho chat IDE & strea
│   │
│   └── tests/                              # Bộ kiểm thử tự động toàn diện (Pytest)
│       ├── __init__.py                     # Khởi tạo tests package
│       ├── test_api_targets_scans.py       # Kiểm thử API thêm/sửa/xóa mục tiêu và tạo lư
│       ├── test_attack_graph.py            # Kiểm thử thuật toán dựng đồ thị tấn công
│       ├── test_compliance_and_sarif.py    # Kiểm thử xuất chuẩn SARIF và đối chiếu tiêu
│       ├── test_copilot_command.py         # Kiểm thử xử lý câu lệnh ngôn ngữ tự nhiên
│       ├── test_e2e_workflow.py            # Kiểm thử luồng trọn gói từ quét đến sinh báo
│       ├── test_execution_engine.py        # Kiểm thử bộ chạy lệnh (Executor) và chốt chặ
│       ├── test_findings.py                # Kiểm thử tạo, lọc và gộp bằng chứng lỗ hổng
│       ├── test_observability.py           # Kiểm thử tính năng đo đếm token và trace LLM
│       ├── test_original_tools.py          # Kiểm thử tương thích các công cụ quét gốc
│       ├── test_rag_chat.py                # Kiểm thử RAG và tích hợp chat AI
│       ├── test_recon_pipeline.py          # Kiểm thử pipeline trinh sát mạng
│       ├── test_recon_planner_original_tools.py# Kiểm thử bộ lập kế hoạch quét
│       ├── test_scan_status_and_evidence.py   # Kiểm thử cập nhật trạng thái quét và thu thậ
│       ├── test_specialist_agents.py       # Kiểm thử hoạt động của từng Sub-Agent
│       ├── test_target_pipeline.py         # Kiểm thử bóc tách và chuẩn hóa mục tiêu
│       ├── test_terminal_api.py            # Kiểm thử Terminal API và cơ chế alias lệnh
│       ├── test_tool_catalog_yaml.py       # Kiểm thử tính hợp lệ cú pháp của 22 file YAM
│       ├── test_tool_registry.py           # Kiểm thử nạp và tra cứu công cụ từ registry
│       ├── test_verification_reports.py    # Kiểm thử sinh báo cáo kiểm thử an ninh
│       └── test_vulnerability_kb.py        # Kiểm thử truy vấn CSDL tri thức lỗ hổng
│
├── docs/                                   # TÀI LIỆU HỆ THỐNG
│   ├── architecture.md                     # Kiến trúc tổng thể hệ thống, luồng dữ liệu &
│   ├── knowledge-base.md                   # Tài liệu về CSDL tri thức (NVD, KEV, CWE) &
│   ├── security-model.md                   # Mô hình an toàn (Guardrails, chính sách Scop
│   ├── target-system.md                    # Tài liệu đặc tả hệ thống mục tiêu & phạm vi
│   └── toolkit.md                          # Hướng dẫn chi tiết sử dụng & bổ sung công cụ
```

```
│
├── frontend/                                  # PHÂN HỆ GIAO DIỆN (React + Tailwind + Vite)
│   ├── index.html                             # File HTML gốc của giao diện người dùng
│   ├── package.json                           # Khai báo thư viện frontend (React, Lucide, T
│   ├── package-lock.json                      # Khóa phiên bản thư viện frontend
│   ├── postcss.config.js                      # Cấu hình PostCSS cho Tailwind CSS
│   ├── tailwind.config.js                     # Thiết lập theme phong cách Cyberpunk/Dark-mo
│   ├── tsconfig.json                          # Cấu hình trình biên dịch TypeScript
│   ├── tsconfig.node.json                     # Cấu hình TypeScript cho môi trường Node/Vite
│   ├── vite.config.ts                         # Cấu hình Vite build tool & proxy chuyển tiếp
│   └── src/                                   # Mã nguồn giao diện
│       ├── App.tsx                            # Component gốc phân chia layout màn hình (Spl
│       ├── index.css                          # Định nghĩa CSS toàn cục và tùy biến thanh cu
│       ├── main.tsx                           # Điểm gắn ứng dụng React vào DOM
│       ├── api/                               # Tầng gọi API Backend từ giao diện
│       ├── common/                            # Các UI components dùng chung (Buttons, Modal
│       ├── hooks/                             # Custom React Hooks (useWebSocket, useTermina
│       ├── lib/                               # Thư viện tiện ích phụ trợ cho giao diện
│       ├── pages/                             # Các trang giao diện chính
│       ├── styles/                            # Các cấu hình style bổ sung
│       ├── types/                             # Khai báo TypeScript types cho frontend
│       ├── store/
│       │   └── workspaceStore.ts              # Quản lý state toàn cục (mục tiêu, chat, logs
│       └── components/                        # Các phân khu giao diện chức năng
│           ├── Chat/
│           │   └── ChatPanel.tsx              # Khung chat tương tác với AI Pentest Copilot
│           ├── Editor/
│           │   └── MainPanel.tsx              # Khung hiển thị nội dung chính đa tab
│           ├── Explorer/
│           │   └── ExplorerPanel.tsx          # Trình quản lý cây thư mục & tệp tin chứng cứ
│           ├── Findings/
│           │   └── FindingsManager.tsx        # Bảng quản lý, lọc và xem chi tiết các lỗ hổn
│           ├── Layout/
│           │   └── Header.tsx                 # Thanh tiêu đề trên cùng (chọn model, trạng t
│           ├── Reports/
│           │   └── ReportViewer.tsx           # Trình xem trước và xuất báo cáo kiểm thử
│           ├── Settings/                      # Bảng điều khiển cấu hình hệ thống
│           ├── Target/
│           │   └── TargetInputModal.tsx       # Cửa sổ nhập mục tiêu mới và thiết lập phạm v
│           ├── Terminal/
│           │   └── TerminalPanel.tsx          # Cửa sổ dòng lệnh tương tác trực tiếp qua Web
│           └── Topology/
│               └── TopologyViewer.tsx         # Trực quan hóa tương tác đồ thị mạng & bề mặt
│
├── infra/                                     # HẠ TẦNG & TRIỂN KHAI HỆ THỐNG
│   ├── devsecops/
│   │   └── .gitlab-ci-pentest.yml             # Pipeline tích hợp DAST vào quy trình DevOps
│   └── docker/
│       ├── Dockerfile.backend                 # Hướng dẫn đóng gói Docker image cho Backend
│       ├── Dockerfile.frontend                # Hướng dẫn đóng gói Docker image cho Frontend
│       ├── Dockerfile.tools                   # Image chứa sẵn toàn bộ công cụ pentest (Nmap
│       ├── docker-compose.tools.yml           # Compose file chạy độc lập cụm container công
│       └── nginx.conf                         # Cấu hình Web Server Nginx chuyển hướng API v
```

```
│
├── scripts/
│   └── init.sh                                 # Bash script kiểm tra môi trường, tạo thư mục
│
├── security/                                   # TIÊU CHUẨN DỮ LIỆU & SCHEMAS AN NINH
│   ├── attack-plans/                           # Các kịch bản tấn công mẫu đã chuẩn hóa
│   ├── datasets/                               # Bộ dữ liệu dùng để benchmark & kiểm thử AI
│   │   ├── expected/                           # Kết quả đầu ra kỳ vọng mẫu
│   │   ├── knowledge/                          # Tri thức mẫu cho kiểm thử RAG
│   │   └── samples/                            # Dữ liệu quét mẫu của các mục tiêu phòng lab
│   ├── docs/                                   # Tài liệu bổ sung về an ninh
│   ├── evaluation/                             # Bộ kịch bản chấm điểm và đánh giá hiệu năng
│   ├── knowledge/                              # Thư mục tri thức an ninh lưu trữ
│   └── schemas/                                # Bộ JSON Schemas chuẩn hóa kết quả đầu ra của
│       ├── cve.schema.json                     # Cấu trúc dữ liệu chuẩn cho lỗ hổng CVE
│       ├── cwe.schema.json                     # Cấu trúc dữ liệu chuẩn cho điểm yếu CWE
│       ├── ffuf_result.schema.json             # Cấu trúc chuẩn kết quả rà quét của FFUF
│       ├── finding.schema.json                 # Cấu trúc chuẩn cho một phát hiện lỗ hổng an
│       ├── gobuster_result.schema.json         # Cấu trúc chuẩn kết quả rà quét thư mục Gobus
│       ├── httpx_result.schema.json            # Cấu trúc chuẩn kết quả thăm dò HTTP của HTTP
│       ├── katana_result.schema.json           # Cấu trúc chuẩn kết quả crawler của Katana
│       ├── masscan_result.schema.json          # Cấu trúc chuẩn kết quả quét cổng Masscan
│       ├── nikto_result.schema.json            # Cấu trúc chuẩn kết quả quét web của Nikto
│       ├── nmap_result.schema.json             # Cấu trúc chuẩn kết quả quét cổng/dịch vụ Nma
│       ├── nuclei_finding.schema.json          # Cấu trúc chuẩn kết quả phát hiện lỗ hổng của
│       ├── proxy_manual.schema.json            # Cấu trúc chuẩn cho thao tác kiểm thử thủ côn
│       ├── rustscan_result.schema.json         # Cấu trúc chuẩn kết quả quét cổng RustScan
│       ├── searchsploit_result.schema.json     # Cấu trúc chuẩn kết quả tra cứu mã khai thác
│       ├── service_enum_result.schema.json     # Cấu trúc chuẩn kết quả liệt kê dịch vụ mạng
│       ├── target.schema.json                  # Cấu trúc chuẩn định nghĩa mục tiêu và phạm v
│       └── whatweb_result.schema.json          # Cấu trúc chuẩn kết quả nhận diện công nghệ W
│
├── shared/                                     # TÀI NGUYÊN DÙNG CHUNG FRONTEND - BACKEND
│   ├── api/                                    # Hằng số định nghĩa endpoint API
│   ├── constants/                              # Các hằng số cấu hình hệ thống
│   ├── websocket/                              # Định nghĩa cấu trúc gói tin qua WebSocket
│   └── types/                                  # Kiểu dữ liệu TypeScript dùng chung
│       ├── scan.ts                             # Kiểu dữ liệu trạng thái và cấu hình lượt qué
│       └── target.ts                           # Kiểu dữ liệu đối tượng mục tiêu kiểm thử
│
└── toolkit/                                    # WORKSPACE THỰC ĐỊA THEO CHU TRÌNH PENTEST
    ├── 00-meta/                                # Chứa mẫu định dạng (templates) và siêu dữ li
    │   └── templates/
    ├── 01-targets/                             # Quản lý mục tiêu kiểm thử
    │   ├── campaigns/                          # Quản lý theo từng chiến dịch kiểm thử
    │   │   └── lab-local/
    │   │       ├── campaign.yaml               # Cấu hình chiến dịch kiểm thử cho phòng lab c
    │   │       └── targets/                    # Các mục tiêu thuộc chiến dịch lab-local
    │   ├── presets/                            # Cấu hình mục tiêu mẫu dựng sẵn
    │   │   ├── vm-dvwa.yaml                    # Mục tiêu máy ảo dễ bị tấn công DVWA
    │   │   ├── vm-juice-shop.yaml              # Mục tiêu ứng dụng dễ bị tấn công OWASP Juice
    │   │   └── vm-metasploitable.yaml          # Mục tiêu máy ảo Metasploitable
    │   ├── registry/
```

```
│   │   └── targets.json                    # File danh bạ đăng ký toàn bộ mục tiêu hiện h
│   └── templates/
│       └── target-ip.yaml                  # File mẫu để khai báo một mục tiêu IP mới
├── 02-passive/                             # Lưu dữ liệu thu thập thông tin thụ động (OSI
├── 03-network/                             # Lưu kết quả quét mạng (Network Scanning)
│   └── scans/
│       └── target-192.168.100.153/         # Thư mục dữ liệu quét của một mục tiêu thực t
│           ├── naabu/                      # Kết quả công cụ quét cổng Naabu
│           │   ├── commands.md             # Lệnh Naabu đã chạy
│           │   ├── raw/naabu_raw.txt       # Kết quả thô chưa qua xử lý
│           │   └── status.json             # Trạng thái hoàn thành quét
│           └── nmap/                       # Kết quả công cụ quét cổng Nmap
│               ├── commands.md             # Lệnh Nmap đã chạy
│               ├── normal/nmap_normal.txt  # Log hiển thị chuẩn màn hình
│               ├── raw/nmap_raw.xml        # Dữ liệu đầu ra định dạng XML thô
│               ├── status.json             # Trạng thái quét
│               └── structured/nmap_result.json # Dữ liệu Nmap đã parse theo chuẩn JSON S
├── 04-web/                                 # Lưu kết quả quét ứng dụng web (Crawling, Fuz
│   └── scans/
├── 05-vulnerability/                       # Lưu kết quả quét lỗ hổng bảo mật tự động (Nu
│   └── scans/
├── 06-service-enum/                        # Lưu kết quả thu thập sâu từng dịch vụ (SMB,
├── 07-cloud-secret/                        # Lưu kết quả dò quét lộ mật khẩu, API key, cấ
├── 08-manual/                              # Lưu vết các phiên kiểm thử thủ công qua Prox
├── 09-analysis/                            # Lưu kết quả phân tích tổng hợp bề mặt tấn cô
│   ├── findings/                           # Các lỗ hổng đã được phân tích và liên kết
│   │   └── 192.168.100.15_findings.json    # Danh sách lỗ hổng tổng hợp của mục tiêu
│   └── manifests/                          # Bản kê khai bề mặt tấn công
│       └── target-192.168.100.153_manifest.json # Bề mặt tấn công đầy đủ của mục tiêu
├── 10-findings/                            # Thư mục lưu trữ bằng chứng lỗ hổng
├── 10-reports/                             # Nơi chứa các báo cáo Pentest đã xuất
│   ├── 192.168.100.153_report.html         # Báo cáo dạng HTML tương tác của mục tiêu 192
│   ├── 192.168.100.153_report.json         # Báo cáo dạng JSON máy đọc được
│   ├── 192.168.100.153_report.md           # Báo cáo dạng Markdown chuyên nghiệp
│   ├── 192.168.100.153_report.sarif        # Báo cáo dạng SARIF phục vụ tích hợp CI/CD
│   ├── 192.168.56.101_report.html          # Báo cáo HTML của mục tiêu 192.168.56.101
│   ├── 192.168.56.101_report.json          # Báo cáo JSON của mục tiêu 192.168.56.101
│   ├── 192.168.56.101_report.md            # Báo cáo Markdown của mục tiêu 192.168.56.101
│   ├── 192.168.56.101_report.sarif         # Báo cáo SARIF của mục tiêu 192.168.56.101
│   ├── target-192.168.56.101-3000_report.html # Báo cáo kiểm thử dịch vụ Web port 3000
│   ├── target-192.168.56.101-3000_report.md   # Báo cáo Markdown kiểm thử web port 3000
│   ├── target-unverified-only_report.sarif # Báo cáo SARIF lọc các phát hiện chưa xác mi
│   └── test-sarif-target_report.sarif      # File SARIF phục vụ kiểm thử hệ thống
├── 11-plans/                               # Lưu các kế hoạch kiểm thử thâm nhập tự động
├── 12-reports/                             # Thư mục phụ trợ lưu trữ báo cáo
├── configs/                                # Thư mục chứa các file cấu hình công cụ mở rộ
├── data/
│   └── kb.sqlite                           # CSDL SQLite chứa toàn bộ tri thức NVD, KEV,
├── logs/                                   # Thư mục lưu trữ log thực thi của hệ thống
├── scripts/                                # Các script tự động hóa chạy công cụ và parse
│   ├── init_dirs.py                        # Script tự động tạo toàn bộ cây thư mục toolk
│   ├── parse_ffuf.py                       # Chuyển đổi output FFUF sang JSON Schema
│   ├── parse_gobuster.py                   # Chuyển đổi output Gobuster sang JSON Schema
```

```
│   ├── parse_httpx.py                # Chuyển đổi output HTTPx sang JSON Schema
│   ├── parse_katana.py               # Chuyển đổi output Katana sang JSON Schema
│   ├── parse_masscan.py              # Chuyển đổi output Masscan sang JSON Schema
│   ├── parse_nikto.py                # Chuyển đổi output Nikto sang JSON Schema
│   ├── parse_nmap.py                 # Chuyển đổi XML của Nmap sang JSON Schema
│   ├── parse_nuclei.py               # Chuyển đổi kết quả Nuclei sang JSON Schema
│   ├── parse_rustscan.py             # Chuyển đổi output RustScan sang JSON Schema
│   ├── parse_searchsploit.py         # Chuyển đổi output SearchSploit sang JSON Sch
│   ├── parse_service_enum.py         # Chuyển đổi output dịch vụ mạng sang JSON Sch
│   ├── parse_whatweb.py              # Chuyển đổi output WhatWeb sang JSON Schema
│   ├── pipeline_runner.py            # Điều phối chạy chuỗi công cụ tuần tự (Pipeli
│   ├── run_ffuf.py                   # Wrapper script chạy an toàn công cụ FFUF
│   ├── run_gobuster.py               # Wrapper script chạy an toàn công cụ Gobuster
│   ├── run_httpx.py                  # Wrapper script chạy an toàn công cụ HTTPx
│   ├── run_katana.py                 # Wrapper script chạy an toàn công cụ Katana
│   ├── run_masscan.py                # Wrapper script chạy an toàn công cụ Masscan
│   ├── run_naabu.py                  # Wrapper script chạy an toàn công cụ Naabu
│   ├── run_nikto.py                  # Wrapper script chạy an toàn công cụ Nikto
│   ├── run_nmap.py                   # Wrapper script chạy an toàn công cụ Nmap
│   ├── run_nuclei.py                 # Wrapper script chạy an toàn công cụ Nuclei
│   ├── run_pipeline.sh               # Bash script chạy toàn bộ pipeline trinh sát
│   ├── run_proxy_manual.py           # Hỗ trợ cấu hình proxy và ghi log kiểm thử th
│   ├── run_rustscan.py               # Wrapper script chạy an toàn công cụ RustScan
│   ├── run_searchsploit.py           # Wrapper script chạy tìm kiếm mã khai thác Se
│   ├── run_service_enum.py           # Wrapper script liệt kê dịch vụ sâu
│   ├── run_whatweb.py                # Wrapper script nhận diện công nghệ WhatWeb
│   ├── scan_status.py                # Cập nhật và truy vấn trạng thái tiến trình q
│   └── targets.py                    # Quản lý danh sách và kiểm tra mục tiêu từ dò
├── tmp/                              # Thư mục chứa dữ liệu tạm thời trong lúc quét
├── tools/                            # Các script wrapper công cụ cấp thấp
│   └── network/
│       └── run_nmap.sh               # Script shell thực thi nmap trong container
└── wordlists/                        # Chứa từ điển phục vụ dò quét (brute-force/fu
```